run-release-canary.sh

§6.Z release-artifact cold-start canary (LVA-077).

Script: `scripts/run-release-canary.sh`

Why this exists

§6.Z mandates that the EXACT artifact about to be distributed is **executed** (not source-compiled) on a real device/VM and observed to survive cold-start **before** distribute. The 1.2.19-1039 forensic anchor was an R8-only crash (`painterResource` rejecting a `<layer-list>`) that the DEBUG variant never hit — proving the release (minified/R8) APK needs its own on-device canary.

`scripts/run-genymotion-challenges.sh` only runs the DEBUG variant via `connectedDebugAndroidTest` (there is no `connectedReleaseAndroidTest` — the app's `testBuildType` is debug). This script fills that gap.

What it does

1. Resolves the Genymotion VM serial via the Containers-submodule genymotion CLI (thin glue, §6.AG — a §6.AH-authorized non-host-direct surface).
2. Installs the exact RELEASE APK from `releases/<version>/android-release/` onto the VM.
3. Cold-launches the app (`pm clear` then launch) and observes the process survives `onCreate` → first frame with NO fatal in logcat.
4. Records the canary outcome as §6.Z evidence under `.lava-ci-evidence/release-canary/`.

Usage

```
./scripts/run-release-canary.sh            # default channel/version from releases/
```

Constitutional bindings

§6.Z (pre-distribute execution proof), §6.AA (release-stage canary), §6.AG/§6.AH (Containers-submodule-driven VM, never host-direct), §6.J (real cold-start observation, not a `podman ps` liveness bluff).